always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deleting a SLA

The process to delete a SLA is exactly as described in the SLA documentation page.

Exception deployment

To perform an exception deployment for SLAs, navigate to the SLAs project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the SLAs.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/ssat.md

{{% alert title="Note" color="primary" %}}

SSAT changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

Changes to these can cause severe impact to KPIs. Exercise extreme caution before proceeding.

{{% /alert %}}

Modifying SSAT settings

You will make the change directly in Zendesk.

SECTION: security/customer-support-operations/workflows/zendesk/themes.md

{{% alert title="Note" color="primary" %}}

All theme changes are handled via a standard deployment.

{{% /alert %}}

{{% alert title="Note" color="danger" %}}

Never directly modify themes in theh Zendesk UI in any instance (sandbox or production). You will severely break the entire setup if you do (and it is not easy to fix).

{{% /alert %}}

Modifying the theme

{{% alert title="Note" color="danger" %}}

This is a very customer facing project. Exercise caution in making changes and ensure you thoroughly reiew a preview of the changes in the sandbox.

{{% /alert %}}

This is done via merge requests to the theme projects.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The process will generate a preview theme for you to review the changes it will make in the UI (for the corresponding sandbox instance). Upon merge, this preview theme will be removed.

Exception deployment

To perform an exception deployment for the theme, navigate to the theme project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the theme.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/ticket-fields.md

{{% alert title="Note" color="primary" %}}

All ticket field changes are handled via a standard deployment.

{{% /alert %}}

Creating a ticket field

For these, you will need to create a corresponding ticket field file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a ticket field

For these, you will need to locate the corresponding ticket field file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deactivating a ticket field

{{% alert title="Note" color="primary" %}}

Be very cautious on these. You can break a considerable amount of things by deactivating a ticket field that is still actually in use.

{{% /alert %}}

To deactivate a ticket field, you must make the following changes in the corresponding ticket field file in the sync repo:

1. Move the ticket field file from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)
1. Change the value of the active attribute in the YAML file to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting a ticket field

{{% alert title="Note" color="danger" %}}

We do not delete ticket fields except during an annual review of our ticket fields. Any other time, leave it as deactivated.

If you are looking for information about the annual review of ticket fields, please see Annual Review workflow

{{% /alert %}}

The process to delete a ticket field is exactly as described in the ticket fields documentation

page.

Exception deployment

To perform an exception deployment for ticket fields, navigate to the ticket fieldsa project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the ticket fields.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/ticket-processor.md

{{% alert title="Note" color="primary" %}}

Ticket processor changes are classified as ad-hoc. Once changes are merged into the default branch, they will be live.

These also usually involve changes to other components we manage.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

SECTION: security/customer-support-operations/workflows/zendesk/tickets.md

{{% alert title="Note" color="primary" %}}

Ticket settings changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

Changes to these can cause severe problems if you have not thoroughly tested them. Exercise extreme caution before proceeding.

{{% /alert %}}

Modifying ticket settings

You will make the change directly in Zendesk. Remember any changes should also be reflected in the tickets documentaiton (as it is the single source of truth).

SECTION: security/customer-support-operations/workflows/zendesk/triggers.md

{{% alert title="Note" color="primary" %}}

All triggers changes are handled via a standard deployment.

{{% /alert %}}

Creating a trigger

For these, you will need to create a corresponding trigger file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a trigger

For these, you will need to locate the corresponding trigger file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deactivating a trigger

To deactivate an trigger, you must make the following changes in the corresponding trigger file in the sync repo:

1. Move the trigger from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)
1. Change the value of the active attribute in the YAML file to false
1. Change the conditions seciton to be:

```yaml
- field: 'brandid'
```

operator: 'isnot'
value: 'GitLab Support'

1. Change the actions section to be:

yaml
- field: 'brandid'
value: 'GitLab Support'

1. Change the value of the containsmanagedcontent attribute to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting a trigger

{{% alert title="Note" color="danger" %}}

We do not delete triggers except during an annual review of our triggers. Any other time, leave it as deactivated.

If you are looking for information about the annual review of triggers, please see Annual Review workflow

{{% /alert %}}

The process to delete a trigger is exactly as described in the triggers documentation page.

Exception deployment

To perform an exception deployment for triggers, navigate to the triggers project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the triggers.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo
- Zendesk Global managed content repo
- Zendesk US Government managed content repo

SECTION: security/customer-support-operations/workflows/zendesk/us-government-exceptions.md

Which organizations sync to the US Government instance by default?

As the exact conditions of this can change at any time, it is best to refer to our

documentation page on the ZD-SFDC sync and the SOQL query documented within.

Adding the exception

When a request is made via the Support Super Form, a series of checks are performed:

- Does the provided requester email correlate to a gitlab.com account?
- Does the requester have a Zendesk US Government agent account?
- Does the SFDC account provided correlate to an actual SFDC account?
- Does the SFDC opportunity provided correlate to an actual SFDC opportunity?
- Does the SFDC account already have the exception?
- Does the SFDC Account have the US Government SKU present on it?

If all checks pass, an issue is created. From here, your task is to add the exception onto the Salesforce account.

To do so, you will navigate to the Swap US Gov Support Exception project and do the following:

1. Navigate to the project's pipelines page
1. Click New pipeline at the top-right of the page
1. Add two variables under the Variables section:
   1. Type: Variable, Input variable key: ACCOUNTID, Input variable value: The SFDC Account's ID
   1. Type: Variable, Input variable key: ISSUEIID, Input variable value: The ID of the issue you are working
1. Click New pipeline

From here, a pipeline is created. Monitor the pipeline to ensure it succeeds correctly.

If you encounter any issues, please notify the Fullstack Engineer so it can be looked into and resolved.

Removing the exception

When a request is made via the Support Super Form, a series of checks are performed:

- Does the provided requester email correlate to a gitlab.com account?
- Does the requester have a Zendesk US Government agent account?
- Does the SFDC account provided correlate to an actual SFDC account?
- Does the SFDC account have the exception?

If all checks pass, an issue is created. From here, your task is to add the exception onto the Salesforce account.

To do so, you will navigate to the Swap US Gov Support Exception project and do the following:

1. Navigate to the project's pipelines page
1. Click New pipeline at the top-right of the page
1. Add two variables under the Variables section:
   1. Type: Variable, Input variable key: ACCOUNTID, Input variable value: The SFDC Account's

ID
   1. Type: Variable, Input variable key: ISSUEIID, Input variable value: The ID of the issue you are working
1. Click New pipeline

From here, a pipeline is created. Monitor the pipeline to ensure it succeeds correctly.

If you encounter any issues, please notify the Fullstack Engineer so it can be looked into and resolved.

 Doing this manually

{{% alert title="Note" color="danger" %}}

This is informational only. This should never be done manually unless an unrecoverable failure has occurred.

{{% /alert %}}

1. Create a file named swapsupportinstanceinsfdc

   bash
   touch swapsupportinstanceinsfdc

1. Make it executable

   bash
   chhmod +x swapsupportinstanceinsfdc

1. Put the following contents into the file:

   ruby
   !/usr/bin/env ruby

   require 'supportreadiness'

   salesforceconfig = Readiness::Salesforce::Configuration.new
   salesforceconfig.apiversion = '58.0'
   salesforceconfig.clientid = ENV.fetch('SFDCCLIENTID')
   salesforceconfig.clientsecret = ENV.fetch('SFDCCLIENTSECRET')
   salesforceconfig.password = ENV.fetch('SFDCPASSWORD')
   salesforceconfig.securitytoken = ENV.fetch('SFDCSECURITYTOKEN')
   salesforceconfig.username = ENV.fetch('SFDCUSERNAME')
   salesforceclient = Readiness::Salesforce::Client.new(salesforceconfig)

   accountid = ARGV.first

   querystring = "SELECT Name, SupportInstancec FROM Account WHERE Id = '{accountid}'"

```
  query = Readiness::Salesforce::Queries.new(querystring)
  results = Readiness::Salesforce::Queries.run!(salesforceclient, query)

  newinstance = if results.first.SupportInstancec == 'federal-support'
                  'gitlab'
                elsif results.first.SupportInstancec == 'gitlab'
                  'federal-support'
                else
                  raise "ERROR: Unknown current instance of
'{results.first.SupportInstancec}'"
                end

  puts "Run info\n"
  puts "- Id:   {accountid}"
  puts "- Name: {results.first.Name}"
  puts "  - Current value: {results.first.SupportInstancec}"
  puts "  - New value:     {newinstance}\n"

  print "Updating to {newinstance}..."
  Readiness::Salesforce::Client.update!(
    salesforceclient,
    accountid,
    SupportInstancec: newinstance
  )
  puts 'done'
```

1. Run the script

```
  bash
  ./swapsupportinstanceinsfdc ABC123DEF456GHI789
```

Output should look like this:

```
bash
jason@laptop:~$ ./swapsupportinstanceinsfdc ABC123DEF456GHI789
Run info
- Id:   ABC123DEF456GHI789
- Name: Test Account
  - Current value: gitlab
  - New value:     federal-support
Updating to federal-support...done
```

Manually update the issue with the results.

SECTION: security/customer-support-operations/workflows/zendesk/user-association.md

{{% alert title="Note" color="primary" %}}

This only applies to Zendesk Global. This should never be used for Zendesk US Government.

{{% /alert %}}

User association is a very specific and particular process. This is for both security and accuracy.

Proving support entitlement

When the request comes in, the we need to first obtain proof of entitlement. This can come in one of three methods:

- The requester is already associated to an organization, in which you can proceed to Adding additonal contacts.
- This is relating to a gitlab.com subscription and the requester is an owner on the top-level namespace, in which you can proceed to Attempt auto-association.
- The requester provides licensing information, in which case you can proceed to Verifying license information.

If none of the above are true, we need to either reject the request or ask for additonal information.

If this is pertaining to a gitlab.com subscription, we need to reject the request. You would do so using the macro Support::Support-Ops::GitLab.com user is not an Owner.

If this is pertaining to a Self-Managed or GitLab Dedicated subscription, we need to request additional information. You would do so using the macro Support::Support-Ops::Proof of support entitlement. If the requester can not provide the needed information, you would need to reject the request.

Pre-checks

Even with support entitlement proven, we need to ensure we can actually associate the user. Before proceeding, make sure all of the followings checks have passed:

- The organization is not using a contact management project
- The addition of the support contacts would not cause the organization to surpass the 30 support contact limit
- There are not organization notes/details indicating you should not proceed with the request

Attempt auto-association

{{% alert title="Note" color="primary" %}}

Before proceeding, please ensure all Pre-checks have been done.

{{% /alert %}}

This is done via the Support Ops Super App. Confirm all metadata on the ticket is correct, refresh your apps, then run the Attempt Association plugin on the app. This will handle associating the user to the correct organization, if possible.

If they wanted additional contacts added, please see Adding additonal contacts.

Handling app failures

If the app failes to auto-associate, it will detail the reason it could not. If it was due to failing one of the Pre-checks, please re-review those and handle the issue as detailed there.

If it is due to complications determining the Salesforce account ID, you will need to manually locate the Salesforce account ID and then proceed to Manual association.

Manual association

{{% alert title="Note" color="primary" %}}

Before proceeding, please ensure all Pre-checks have been done.

{{% /alert %}}

The steps for manual association are as follows:

1. Locate the organization
1. Copy the name of the organization
1. Go to the user in question
1. Paste the organization name into the Org field (top-left)
1. Click the organization that appears in the drop-down

After doing so, if they wanted additional contacts added, please see Adding additonal contacts.

Adding additional contacts

{{% alert title="Note" color="primary" %}}

Before proceeding, please ensure all Pre-checks have been done.

{{% /alert %}}

To add additional contacts, you should use the Associate User plugin in the Support Ops Super App.

If the app failes to associate, you will need to utilize Manual association.

Verifying license information

When a Self-Managed or GitLab Dedicated customer provides us thier proof of entitlement, we should have two key pieces of information:

- The subscription owner's email (i.e. the sold-to)
- The license information

The path forward will depend on exactly what the license information is, but your first step is to locate the license of cloud activation page.

Locating the license or cloud activation page

- If given a license ID:
  1. Navigate to the customers.gitlab.com admin panel
  1. Go to the Licenses page
  1. Add /xxxx to the end of your URL (replacing xxxx with the license ID)
- If given a raw license file:
  1. Navigate to the customers.gitlab.com admin panel
  1. Go to the Licenses page
  1. Go to the Validate License page
  1. Paste the contents of the license file in the box
  1. Click Validate
  1. Copy the id value (at the bottom of the page)
  1. Go to the Licenses page
  1. Add /xxxx to the end of your URL (replacing xxxx with the license ID)
- If given a license usage export:
  1. Open the file
  1. Copy the raw License Key value
  1. Navigate to the customers.gitlab.com admin panel
  1. Go to the Licenses page
  1. Go to the Validate License page
  1. Paste the contents of the license file in the box
  1. Click Validate
  1. Copy the id value (at the bottom of the page)
  1. Go to the Licenses page
  1. Add /xxxx to the end of your URL (replacing xxxx with the license ID)
- If given a cloud activation code:
  1. Navigate to the customers.gitlab.com admin panel (to ensure you are logged in)
  1. Navigate to https://customers.gitlab.com/admin/cloudactivation?query=XXXX (replacing XXXX with the cloud activation code)
  1. Open the found cloud activation

Once you have the license or cloud activation itself pulled up, make an internal note on the ticket with the link (to save others time in future lookups).

You will then verify the following:

- The license is not a trial license
- The license is still active (i.e. has not expired yet)

If either of those fail, it is not a valid license to use for proving support entitlement (and you should reply to the ticket stating we need an active non-trial license).

If there are not issues, you next need to locate the order.

Locating the order

To do this:

1. Copy the Zuora subscription name value from the license page
   - If on a cloud activation page, the value to copy is actually Subscription name
1. Go to the Orders page
1. Click Add filter as the top-right of the page
1. Click Subscription name (this cases a filter bar to appear at the top-left of the orders list)
1. Click the drop-down and select Contains
1. Put the value of the Zuora subscription name in the input box (to the right of the drop-down)
   - Make sure you didn't accidently copy extra spaces!
1. Click thhe Refresh button (or hit Enter on your keyboard)

This should pull up order in the list of results. Navigate to it and make an internal note on the ticket with the link (to save others time in future lookups).

From here, you need to locate the billing account

Locating the billing account

Being on the order page, this should be simply, as it should be a clickable link on the order page. Navigate to it and make an internal note on the ticket with the link (to save others time in future lookups).

On this final page, we have the last two pieces of information we need:

- The Salesforce account
- The Sold to

Verify the sold-to matches what the requester provided us. If it does not, we cannot proceed.

If all checks out, you can proceed to Locating the organization.

Locating the organization

Once everything has been verified, you are ready to find the organization. To do this, perform a Zendesk search:

1. Click the hourglass icon within Zendesk (top-rightish of the page)
1. Type salesforceid:XXXX (replacing XXXX with the Salesforce account value you located)
1. Click the Organizations tab
1. Open the found organization

In the rare event the organziation seems to not exist, make an internal note indicating as much and assign the ticket to the Fullstack Engineer to review.

Locating the SFDC account ID

In cases where the Attempt Association app failed to locate the correct Salesforce account ID, you will need to locate it manually.

To do this, take the top level paid namespace in question and use it via the Namespace Lookup plugin on the Support Ops Super App. Review the results it provides to determine the correct Salesforce account ID to use.

Sending out the final reply

Once you have completed all association, make sure to send out a reply and mark the ticket as Solved. The macro Support::Support-Ops::Users added to organization can be used as a reference for the reply.

SECTION: security/customer-support-operations/workflows/zendesk/user-fields.md

{{% alert title="Note" color="primary" %}}

All user field changes are handled via a standard deployment.

{{% /alert %}}

Creating a user field

For these, you will need to create a corresponding user field file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Editing a user field

For these, you will need to locate the corresponding user field file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Deactivating a user field

{{% alert title="Note" color="primary" %}}

Be very cautious on these. You can break a considerable amount of things by deactivating a user field that is still actually in use.

{{% /alert %}}

To deactivate a user field, you must make the following changes in the corresponding user field file in the sync repo:

1. Move the user field file from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)
1. Change the value of the active attribute in the YAML file to false

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting a user field

{{% alert title="Note" color="danger" %}}

We do not delete user fields except during an annual review of our user fields. Any other time, leave it as deactivated.

If you are looking for information about the annual review of user, please see Annual Review workflow

{{% /alert %}}

The process to delete a user field is exactly as described in the user fields documentation page.

Exception deployment

To perform an exception deployment for user fields, navigate to the user fieldsa project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the user fields.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo

SECTION: security/customer-support-operations/workflows/zendesk/views.md

{{% alert title="Note" color="primary" %}}

All views changes are handled via a standard deployment.

{{% /alert %}}

Creating a view

For these, you will need to create a corresponding view file in the sync repo.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Editing a view

For these, you will need to locate the corresponding view file in the sync repo and make changes to it.

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

The exact nature of what you need to put into the YAML file will vary based on the issue's request.

Be sure to read Working with sync repo files for more information.

Deactivating a view

{{% alert title="Note" color="primary" %}}

Due to the nature of the multi-step process, this will result in a pipeline failure when step 1 is completed. This will be fixed in step 2.

{{% /alert %}}

To deactivate an view, you must perform a two-step process

Step 1

1. Locate the managed content file for the view
1. Move the file from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)

You should also do this in a way that creates a MR. You will require someone from Support Leadership to approve this MR before you can merge it.

Only proceed to step 2 once all changes in step one are merged.

Step 2

1. Locate the YAML file for the view in the sync repo
1. Move the view from the active folder to the the corresponding location in the inactive folder (if it does not exist, create it)

1. Change the value of the active attribute in the YAML file to false
1. Change the conditions section to be:

```
yaml
- field: 'brandid'
  operator: 'isnot'
  value: 'GitLab Support'
 - field: 'status'
   operator: 'lessthan'
   value: 'closed'
```

1. Change the restriction value to be null

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Deleting a view

{{% alert title="Note" color="danger" %}}

We do not delete views except during an annual review of our views. Any other time, leave it as deactivated.

If you are looking for information about the annual review of views, please see Annual Review workflow

{{% /alert %}}

The process to delete a view is exactly as described in the views documentation page.

Exception deployment

To perform an exception deployment for views, navigate to the views project in question, go to the scheduled pipelines page, and click the play button. This will trigger a sync job for the views.

Repo links

- Zendesk Global sync repo
- Zendesk US Government sync repo
- Zendesk Global managed content repo
- Zendesk US Government managed content repo

SECTION: security/customer-support-operations/workflows/zendesk/webhooks.md

{{% alert title="Note" color="primary" %}}

Webhook changes are classified as ad-hoc as the changes are made directly in Zendesk. Once changes are made in Zendesk they are live.

{{% /alert %}}

Creating a webhook

You will make the change directly in Zendesk. See the webhook documentaiton for more information.

Editing a webhook

You will make the change directly in Zendesk. See the webhook documentaiton for more information.

SECTION: security/customer-support-operations/workflows/zendesk/working-tickets.md

{{% alert title="Note" color="primary" %}}

This pertains solely to Operations working tickets. It does not reflect how any other team might work them.

{{% /alert %}}

Requests to be associated

Global

Please see User Association for more details on associating users to organizations.

US Government

All support contact association is done via a sync with the Salesforce account. As such, we cannot assist here. You will need to instruct the requester to contact their account manager via a new emial. Make sure to provide the email of the account manager.

Once you have done so, mark the ticket as Solved.

Shared organization requests

Please see Shared Organization management for more details on shared organizations.

Contact management project

Please see Contact Management Projects for more details on contact management projects.

Support portal issues

These are reports of issues within the support portal. While each issue can present unique challenges, the common troubleshooting guide for the users would be:

1. Ensure your browser is allowing third party cookies. These are often vital for the system to work. A general list to allow would be:
   - [.]zendesk.com
   - [.]zdassets.com
   - [.]gitlab.com
1. Disable all plugins/extensions/addons on the browser.
1. Disable any themes on the browser.
1. Clear all cookies and cache on the browser.
1. Try logging in again to the Support Portal.
1. If you are still having issues, obtain the following:
   - the browser's version
   - the browser's type
   - your operating system (and distro)
   - any other identifying information
   - the complete contents of your Javascript console for your browser
1. Send all of that to support

At the point you get the ticket, the user may or may not have done all of that. If they have not, point them to trying all that out first.

If they have, you will need to analyze the details of what is sent to determine next steps.

Adding Secondary Email

Sometimes a customer will raise issue stating they want to add a secondary email to their support portal account. Secondary emails are used to tie submitted tickets to a specific account, although only the primary email address will be used as the submitter (and thus receive notifications).

To add a secondary email to a support portal account, we need following information:

- For GitLab.com Users:
  - The GitLab.com account associated to the requester's email address should have listed the secondary email as verified. You can check this via the User Lookup app. To add secondary email to GitLab.com account, they can follow this documentation
- For Self Managed and GitLab Dedicated Users:
  - The ticket needs to be submitted from the email address they wish to have added to their existing profile
  - The customer will need to provide proof of support entitlement again via this secondary email address
  - The customer will need to CC the primary email address of the support portal account and have that email reply on the ticket confirming the request to add the secondary email address to their support portal account.
- For Partners:
  - Use the same steps you would for Self Managed and GitLab Dedicated Users

Incorrect form tickets

When a ticket is using the incorrect form, agents will use the General::Forms::Incorrect form used macro. This will change the form to Support Ops, tag the ticket, and leave an internal note. From there, we are expected to review the ticket and determine the next steps.

Your goal here is to move it to the correct form. Make sure you fill out all ticket metadata possible. When reviewing the ticket metadata, ensure that you remove the assignee unless it's explicitly clear that it should remain with the current assignee. Tickets that remain assigned when moved between forms may be hidden from views and can cause tickets to breach their SLA.

If the ticket stage is set to NRT, but it makes more sense to treat the ticket as a FRT ticket, change the ticket stage to that of FRT.

Handling malicious users

When a ticket arises containing potential malicious actions (hacking, phishing, abuse, etc.), we need to always treat it seriously.

If, after a thorough investigation, it is determined to be malicious, take the following actions:

- Suspend the user
- Add a user note stating the ticket and reason
- Add a rejection for the user's email End-user settings Blocklist (e.g. reject:alice@example.com)
- Close the ticket (via the API)
- Delete any user sessions for the user in question (via the API)

When in doubt, escalate the matter to your manager, a Customer Support Operations Fullstack Engineer, and/or the GitLab Secuirty team.

SECTION: security/customer-support-operations/workflows/zendesk/zd-sfdc-sync.md

{{% alert title="Note" color="danger" %}}

ZD-SFDC Sync changes are classified as ad-hoc. Once changes are made in the repo, they are then used on future syncs.

We rarely directly edit the ZD-SFDC Sync repo itself, as the gitlabsupportreadiness gem does most of the heavy lifting here.

Changes to these can cause severe problems if you have not thoroughly tested them. Exercise extreme caution before proceeding.

{{% /alert %}}

Is a cache reset required?

When making changes to the ZD-SFDC Sync, you must determine if a cache reset is going to be

required. This is determined by the nature of what exactly is being changed. If anything was added, modified, or removed from the redis objects used (in the gitlabsupportreadiness gem), you will have to do a cache reset.

If you are not directly modifying those objects, a cache reset is not required.

Changes involving cache resets

Because a cache reset is required, you need to undergo a very specific (and intensive) process:

1. Disable the scheduled pipeline(s) in the corresponding ZD-SFDC Sync project
1. Clear the redis cache for the corresponding objects
   - This is done by manually running the corresponding scheduled pipeline in the corresponding maintenance project. See our Zendesk maintenance documentaiton for more information
1. Apply the changes (via the repo and/or gem)
1. Run the sync tasks via CLI on your laptop (running in a screen).
   - This is going to take some time depending on the number of changes being made.
   - It is likely you may need to dedicate your whole shift to just doing this task in some cases.
   - Plan accordingly
1. Once all changes are synced, re-enable the scheduled pipeline(s) in the corresponding ZD-SFDC Sync project

Changes not involving cache resets

For these, you will make the changes in the repo (and/or gem).

You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Need to prevent changes from going live before a set date?

While rare, we sometimes need changes to the ZD-SFDC Sync to only go live on a specific date. Being an ad-hoc deployment, this does really enable that naturally.

For situations that require it, make a MR that modifies the Gemfile of the project to lock the gitlabsupportreadiness gem verison used to a specific version.

As an example, if you are needing to lock the version to 1.0.128, the Gemfile will look like this:

```
ruby
 frozenstringliteral: true

source 'https://rubygems.org'

gem 'gitlabsupportreadiness', '1.0.128', require: 'supportreadiness'
```

Once it is the desired deployment , simply chahnge the Gemfile back to the original value,

which should be:

ruby
 frozenstringliteral: true

source 'https://rubygems.org'

gem 'gitlabsupportreadiness', require: 'supportreadiness'

SECTION: security/identity/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

{{% alert title="Not Live Yet" color="warning" %}}
You are viewing a preview of documentation for the future state of GitLab Identity v3 (mid 2024). See the Access Management Policy for the GitLab Identity v2 current state with baseline entitlements and access requests. See the roadmap in the <a href="https://gitlab.com/groups/gitlab-com/gl-security/identity/eng/-/roadmap?state=all&sort=startdateasc&layout=QUARTERS&timeframerangetype=THREEYEARS&grouppa
tlab-com/gl-security/identity/eng&progress=WEIGHT&showprogress=true&showmilestones=false&milest
onestype=ALL&showlabels=true">epics gantt chart.
{{% /alert %}}

{{% alert title="Quick Reference User Guides" color="info" %}}
This page focuses on the architecture and documentation of our Identity Platform and Security Program. If you are looking to get access an application or infrastructure, see the Team Member User Guide. To manage access for team members that report to you, see the Manager User Guide. You can scroll to the bottom of this page to see the other user guides.
{{% /alert %}}

 Mission

GitLab is one of the stewards of the world's source code and intellectual property. Our mission is to ensure that internal and administrative access to customer and product data is protected and industry trust is preserved.

 Risks and Process Improvements

Our work focuses on the following risks and process improvement areas:

1. Lateral Movement Risk: Restricting and securing access to GitLab's internal data and systems based on "need to know" and least privilege security principles to prevent data leakage or data loss.
1. Penetration Risk: Implement and manage security boundaries ("castle walls") related to authentication, authorization, device trust, and least privilege for GitLab team members,

temporary service providers, and service accounts that have access to internal GitLab systems and/or administrative access.
1. Cloud Infrastructure Control Plane: We manage the top-level access and architecture for AWS and GCP cloud providers to enforce least privilege and separation of accounts and resources for workloads.
1. Last Mile Process Automation: Implement custom code for vendor feature gaps for last mile compliance and provisioning automation that improve business efficiency and change management auditability. This improves back office and team member end user experience, automation, and audit reports with onboarding, career mobility, offboarding, and ad-hoc access requests using improved role-based access control architecture.
1. Configuration State Management: Use GitOps configuration/infrastructure-as-code for system configuration where feasible to avoid risks with manual configuration and drift detection.

Future State Goals

Provisioning and Approvals

As we look ahead, we have a North Star vision that access and permissions are provisioned programatically, not manually by system administrators.

We also want to ensure that all approvals are systematic in nature to allow automated audit log exports. For example:

1. A GitLab issue Markdown checkbox is visually auditable, but not programatically auditable. Anyone can select the box so manual effort is needed to verify that the appropriate person checked the box.
1. A GitLab label is auditable if it exists, but not who applied it and timestamp of application. Although the UI shows which user applied the label, this isn't easily accessible in the event log. Any user can apply the label, so there are risks with assurance of who had the authority to apply the label.
1. When we use a Slack bot or Web UI form with a green and red button, the user is already authenticated and only that user has the ability to click that button based on policies that our systems define. We use API calls to perform actions that have additional validation checks and audit logs for tracking.

We want to provide a streamlined user experience for admins, managers, and end users alike to reduce friction and provide self service wherever possible. We have had success with past projects and want to bring our experience and lessons learned to the next generation of Identity Management.

> "I created an AWS account with gitlabsandbox.cloud today. To be honest, I did not expect it to be fully automated. I got my AWS credentials in 5 minutes without bothering anyone. That's amazing!" - Dmitriy Zaporozhets (DZ), GitLab Co-Founder, 2020-12-14

We will be using SaaS vendors (Google Workspace, Okta, Okta IGA, Workday) where possible and providing last mile automation, auditing, and user experience with our custom Identity Platform.

Auditability

We perform manual visual audits in Identity v2 with screenshots and CSV exports.

In Identity v3, we will use programmatic provisioning so every action and metadata diff is logged with a standardized schema for improved alerting, auditability, and dispatching automation for provisioning and deprovisioning. This also improves transparency for team members to self service the information they need and streamlines our user access review process with a history of point-in-time records of what changed, when, and automates any observations.

Elevated, Just-in-Time, and Admin Access

Our next-gen architecture will allow us to automate just-in-time access for elevated and administrative actions that are burdensome to manage manually.

For users that need perpetual administrative access, we have an additional admin account for each of the users in Customer Support, Infrastructure, IT, Security and other roles. See access level wristbands to learn more.

We are also investing in additional administrative control plane separation. Details are not published in the public handbook for security reasons.

Roadmap

As any company grows, they go through phases of growth with many organic iterations throughout the journey. GitLab is currently outgrowing Identity v2 and is building our Identity v3 program.

GitLab Identity v1

GitLab Identity v1 was managed using Tech Ops practices by the Infrastructure and People Operations team prior to 2018.

GitLab Identity v2

GitLab Identity v2 is what we do today and have been doing since 2018 with baseline entitlements and access requests. See the Access Management Policy to learn more.

The processes that we do today meets audit and compliance requirements, however the processes are mostly manual that results in internal inefficiency. It takes a lot of labor hours to manage onboarding, access requests, access reviews, and offboarding processes.

You can see historical stats on access requests in this (internal) spreadsheet.

GitLab Identity v3

{{% alert title="Early Development" color="warning" %}}
We are in the architecture and early engineering incubation phase. Please continue to use existing Identity v2 processes (business as usual) for all requests through mid-2024.
{{% /alert %}}

GitLab Identity v3 is where we want to be in FY25-2H (mid-late 2024) with a pseudo-greenfield approach to automate all of our policies and as much of our approvals, provisioning, and access reviews as possible.

Role Based Access Control (RBAC)

With the introduction of Identity Roles and Identity Groups, we can reduce the number of ad-hoc access requests by using predefined policies and automated provisioning based on role-based access control rather than named user access control.

If you are familiar with job families in Identity v2, this is the next generation with an improved architecture and schema for IAM and RBAC.

Identity Governance (IGA) Implementation

Identity Governance and Administration (IGA) vendor products are typically focused on a compliance lens, with the goal of managing the approval process when a user requests access to an application, and providing a UI for compliance team members to perform user access review audits for existing users that have access to an application.

We are in the early stages of implementing Okta's IGA platform. You can learn more in the internal slide deck and materials in the Google Drive folder. You can also see additional feedback from team members during the RFP process in it/engops289.

The high level goal is that Okta IGA provides an improved user experience for users to request access to Okta applications with a Web UI and Slack Bot that are not managed by baseline entitlements, and has helpful compliance backend UI features for performing user access reviews.

Custom Code

The Security department is leveraging vendors that specialize in Identity Management (ex. Okta, IGA solution, etc.), however no SaaS vendor has all of the features that we need.

Due to the existential and security risks associated with IAM/RBAC, the Security department is supporting our Identity Engineers who are building in-house infrastructure, scripts, and software integrations to build the last mile automation and tools.

The Identity Platform provides the API integration and plumbing of our vendor solutions and allow us to meet the challenges of the future, make GitLab easier to do business with internally, and unilaterally address our IAM and RBAC risks with a holistic approach.

The Identity Platform is a well architected library of scripts that use CI/CD pipeline jobs to parse YAML policy files, generate user manifests, and use the REST API for each respective vendor to check if the user belongs to the group, should be removed from the group, and sync the group members against our policy. See the full data flow for more details.

We will be using GitOps (a.k.a. Terraform and GitLab CI/CD pipelines) approach for assigning groups to Okta applications with state management.

Complex Systems

We have identified several key systems and applications that need our focus and investment to optimize Identity Security. We are focusing on managing the admin control plane and tech stack systems that are causing a lot of business inefficiencies related to onboarding and offboarding.

We describe these as the "complex" systems that don't have easy button provisioning that 3rd party vendors can easily manage. We have to build custom automation using Terraform or REST API calls to handle the complex systems.

- (New) Admin Control Plane with GitOps and Admin Accounts
- GitLab SaaS Instance Admins
- Google Cloud
- Google Workspace Groups
- GitLab SaaS Groups and Members (ex. gitlab-com, gitlab-org, gitlab-, gl- namespaces)
- Okta Group Membership Policies/Rules (used by Okta Applications)

We are also closely involved with processes and systems that cause the most inefficiency or pain and suffering.

- Onboarding issues and access requests
- Baseline entitlement architecture and automation
- Offboarding issues and access deprovisioning
- Okta architecture
- Google Workspace architecture
- 1Password administration
- Google Cloud architecture

For more details, see the related handbook pages, reach out to security-identity-ops with questions, or schedule a call with Jeff Martin.

Automation Options

This is a low context simplified explanation.

Okta Authentication

A large number of our tech stack applications have authentication federated by Okta single-sign on (SSO) using any combination of SCIM, SAML, and OIDC protocols.

It is important to keep in mind that Okta is not an authorization platform. In other words, when you sign in, the application can see your name, email address, and profile attributes or a list of group names that you are a member of, however Okta cannot provision/grant you roles or permissions on the application itself.

Most applications have "simple" provisioning that requires adding the user and they are assigned baseline permissions and users do not need additional permissions. For discussion purposes, this covers ~80% of our applications.

A few applications have code logic with their Okta authentication integration that allows you to specify which user profile attributes or group names should be used to grant access to additional roles and permissions inside of the application, however we rarely see this functionality in applications. Most applications provide a UI that lets you manually assign a role to the existing user.

Authorization Automation

For ~20% of applications that require "resource" or "role" provisioning, we have to use no-code or our own scripts to achieve this. There are no features in Okta or other Identity solutions that offer these features. This requires using the vendor's REST API with our own scripts to call endpoints respectively. Any integrations that you see advertised are usually Professional Services custom integrations that their engineers wrote scripts for.

You can see the GitLab product Members API endpoint as an example. We can force users to sign in with Okta, however Okta cannot automate which groups and projects the user has access to or the role/permission level for each of those groups or projects.

No Code Automation

You may have heard of no code solutions like Okta Workflows, Tines, Workato, Zapier, etc that can solve these needs. While they work reasonably well in simple use cases for users who do not have code experience, they are not the tool for the job in all cases, particularly complex ones.

Remember that no-code workflows are point-to-point and are not usually part of a shared ecosystem that might provide economies of scale. In other words, you can't call a function/method from another class.

With any WYSIWYG tool, you cannot use expression language syntax natively like you can when writing a few lines of code, so you have to catch data, pull an attribute, then plug that string attribute into the next function. The end result is that no-code workflows become overly complicated and can become 50 steps that can be accomplished with 5 lines of code.

In other words, no-code is great for something very simple, but should not be relied on for complex workflows without good reason. In Identity v2, we've tried to stretch the limits of no-code workflows, and believe that scripts running in CI/CD pipelines offer easier maintenance if well architected.

Custom Code Automation

The concept of writing code or custom applications can be perceived as burdensome and requires significant investment. Our strategy is to use small scripts that are run with GitLab CI/CD pipelines, allowing us to keep the code base small and dogfooding the GitLab product while maintaining the smallest possible codebase footprint for automation.

See the Identity Platform to learn more about our approach.

Separation of Concerns

Tech Stack Kingdoms

We have refactored our tech stack into Identity Kingdoms (analogous to a realm) to provide separation of concerns between Business, Cloud, and Product (SaaS and Dedicated) unique needs, particularly with administrative control planes and least privilege configuration. This allows us to create automation and policies specific to each kingdom's compliance requirements to enable the respective teams to operate efficiently within our top-level architecture and guardrails.

Learn more on the Identity Kingdoms handbook page.

Trust Landscape DRIs

The Security team focuses on customer, compliance, and product trust, while the Business Technology and IT team focuses on corporate and financial trust.

- Administrator Trust: Security Department - Identity Ops
- Automation Trust: Security Department - Identity Engineering
- Boundary Trust: Security Department - Identity Engineering
- Compliance Trust: Security Department - Commercial Compliance Team
- Customer Trust: Engineering, Product, and Security division
- Financial Trust: Business Technology/IT Department
- Product Feature Trust: Development Department and Product Management
- Product Reliability Trust: Infrastructure Department

Technological Trust

In addition to maintaining human confidence, we also need to ensure supply chain confidence and that no single vendor breach can compromise the integrity of our architecture, particularly the boundaries that we refer to as castle walls. By using a defense-in-depth approach, we are able to minimize the external attack surface.

Learn more on the Identity Boundaries handbook page.

Data Sources

We consider Workday to be the source of truth for team members. All users and their attributes are synced with Okta every hour with built-in vendor integrations.

The Temporary Service Provider process is the SSOT for contractors and external users. The IT team manages automation that creates temporary service provider users in Okta with -ext@gitlab.com email addresses.

Workday has department, job title, and manager metadata, but does not have sufficient sub-department/team/role metadata that is needed for RBAC (that are being evaluated). Workday also does not have any of our temporary service provider contractors. In our current iteration, we consider Workday to be focused on People Group related use cases.

We have flexible control of custom attributes in Okta that we can pull (and push) so we consider Okta Users and their attributes to be the aggregated SSOT for Identity.

When a user's Okta status changes to deprovisioned (pushed from Workday), we consider them to be offboarded.

The Identity Team aggregates the information from each team to create a consolidated hierarchy of teams that are used for role-based access control and add them to our policy files that are used to generate manifests. See Identity Roles for a full list of data sources and policies.

mermaid
graph TB

subgraph "Workday"
WORKDAYWORKER[Worker]
WORKDAYREGION([Region])
WORKDAYDEPARTMENT([Department])
WORKDAYDIVISION([Division])
WORKDAYJOBTITLE([Job Title])
WORKDAYDEPARTMENT -.- WORKDAYWORKER
WORKDAYDIVISION -.- WORKDAYWORKER
WORKDAYREGION -.- WORKDAYWORKER
WORKDAYJOBTITLE -.- WORKDAYWORKER
WORKDAYWORKER -. "Manager Relationship" .-> WORKDAYWORKER
end

subgraph Identity SaaS Vendor Services
direction LR
subgraph Okta GitLab Tenant
OKTAPROFILEMAPPING{{Workday to Okta
Profile Mapping}}
IDENTITYVENDOROKTAAPIUSERS[("Okta Users")]
IDENTITYVENDOROKTAAPIENDPOINT[("Okta API
https://gitlab.okta.com/api/v1")]
WORKDAYWORKER -- "SCIM Provisioning
Every Hour" --> OKTAPROFILEMAPPING
OKTAPROFILEMAPPING --> IDENTITYVENDOROKTAAPIUSERS
IDENTITYVENDOROKTAAPIUSERS --> IDENTITYVENDOROKTAAPIENDPOINT
end
end

classDef slate fill:cbd5e1,stroke:475569,stroke-width:1px;
classDef red fill:fca5a5,stroke:dc2626,stroke-width:1px;
classDef orange fill:fdba74,stroke:ea580c,stroke-width:1px;
classDef yellow fill:fcd34d,stroke:ca8a04,stroke-width:1px;
classDef emerald fill:6ee7b7,stroke:059669,stroke-width:1px;
classDef cyan fill:67e8f9,stroke:0891b2,stroke-width:1px;
classDef sky fill:7dd3fc,stroke:0284c7,stroke-width:1px;
classDef violet fill:c4b5fd,stroke:7c3aed,stroke-width:1px;
classDef fuchsia fill:f0abfc,stroke:c026d3,stroke-width:1px;

Workday Fields

Here is a non-exhaustive list of Okta profile fields that we use for identity management.

- Okta ID
- Name
- Email
- Manager Email (managerId)
- Cost Center (costCenter)
- Division (division)
- Department (department)
- Job Title (title)
- Hire Date (overlaps with createdat usually)
- Management Level (workdaymanagementLevel)
- Region (Americas, EMEA, APAC)
- GitLab SaaS Username
- GitLab SaaS ID
- Slack ID

We have additional fields that we could use but choose not to for PII risk or irrelevant field reasons. This is a two way door decision on an attribute-by-attribute basis.

- Corporate Entity (gitlabloc)
- Employee Number
- Workday Job Code
- Workday ID (similar to Employee Number)
- Postal Address
- City
- State/Province
- Postal Code
- Mobile Phone Number
- Country

User Array Fields

The Identity Platform automation uses our centralized (single) API call to Okta every hour to extract users that are used as the data source for additional automation.

```
yml
dmurphy:
  providerid: 00u1b2c3d4e5fg7h8357
  anonymized: false
  type: purple
  firstname: Dade
  lastname: Murphy
  fullname: 'Dade Murphy'
  email: dmurphy@gitlab.com
  handle: dmurphy
  manager: klibby
  idrole: secidentityeng
  organizationname: null
  costcenter: rd
  division: security
  department: security
```

```
title: securityengineer
managementlevel: individualcontributor
region: americas
gitlabsaasid: 12345678
gitlabsaasusername: z3r0c00l-example
slackid: UA1B2C3D4
status: active
createdatdate: 'YYYY-MM-DD'
createdattimestamp: 'YYYY-MM-DDTHH:MM:SS.000000Z'
createdatagedays: 123
lastloginatdate: 'YYYY-MM-DD'
lastloginattimestamp: 'YYYY-MM-DDTHH:MM:SS.000000Z'
lastloginatagedays: 2
passwordupdatedatdate: 'YYYY-MM-DD'
passwordupdatedattimestamp: 'YYYY-MM-DDTHH:MM:SS.000000Z'
passwordupdatedatagedays: 123
profileupdatedatdate: 'YYYY-MM-DD'
profileupdatedattimestamp: 'YYYY-MM-DDTHH:MM:SS.000000Z'
profileupdatedatagedays: 123
statusupdatedatdate: 'YYYY-MM-DD'
statusupdatedattimestamp: 'YYYY-MM-DDTHH:MM:SS.000000Z'
statusupdatedatagedays: 123
```

Identity Team Functions

The Identity Engineering team was formed in the Security Threat Management sub-department with a cross-department realignment on 2023-12-01 to lead the design and implementation of our next-generation of identity and access management framework and program at GitLab that has been branded as Identity v3 that will be released iteratively throughout 2024.

The Identity Team has three functional specialties and collaborates cross-functionally with our stable counterparts.

Identity Infrastructure Engineering

The Identity Infrastructure team is focused on our top-level cloud provider infrastructure organization-level management for AWS and GCP in collaboration with the Infrastructure Security team.

We build self-service tools for users to create their own AWS accounts and GCP projects and provide castle wall hardening.

Each team that deploys infrastructure resources is responsible for managing their own infrastructure workloads and DevOps operations using industry best practices. In other words, the Security team creates the scaffolding for your castle and provides hardened castle walls, while your team is responsible for anything you build inside the castle walls.

See the Identity Infrastructure handbook page to learn more.

- Slack Channel: security-identity-ops
- Slack Tag: @security-identity
- GitLab Tag: @gitlab-com/gl-security/identity/infra
- Epics: gitlab.com/gl-security/identity/infra
- Issue Tracker: gitlab.com/gl-security/identity/infra/issue-tracker
- Escalation DRI: Jeff Martin or Vlad Stoianovici

Identity Operations

When we operationalize GitLab Identity v3 in mid-2024, we will have cross-functional team members from IT Operations, People Operations, and Security Operations teams that will perform day-to-day administration of Okta, Okta IGA, and the Identity Platform and handle business and user requests.

In the interim, the Identity Platform Engineering and Identity Infrastructure team members provide coverage in collaboration with our counterparts.

- Slack Channel: security-identity-ops
- Slack Tag: @security-identity
- GitLab Tag: @gitlab-com/gl-security/identity/ops
- Epics: gitlab.com/gl-security/identity/ops
- Issue Tracker: gitlab.com/gl-security/identity/ops/issue-tracker
- Escalation DRI: Jeff Martin

Identity Platform Engineering

The GitLab Identity Security team has CI/CD script and fullstack development skills to build our own open source policy and provisioning automation for our IAM and RBAC needs to supplement our vendors and provide last mile automation "plumbing".

You may already use GitLab Sandbox Cloud that is powered by HackyStack. We are refactoring HackyStack to become a component of the Identity Platform.

We are in the early incubation stage of building the Identity Platform that is powered by accessctl, a new open source project. As our Identity Platform matures throughout 2024, we will add documentation for the community to adopt our platform and best practices.

Real-Time Updates

- Slack Channel: security-identity-eng
- Slack Tag: @security-identity
- GitLab Tag: @gitlab-com/gl-security/identity/eng
- Epics: gitlab.com/gl-security/identity/eng
- Issue Tracker: gitlab.com/gl-security/identity/eng/issue-tracker
- Escalation DRI: Jeff Martin

Transparency

Due to the nature of the risks that we mitigate, our roadmap is only visible to internal team members in our epics roadmap.

You can learn more about our progress in our Identity Engineering epics and Identity Engineering issues.

We use public issue trackers for platform feature requests, however all issues related to GitLab's business, data, infrastructure, and risks are created and managed in the Identity Engineering (internal) issue tracker and are linked in open source project merge requests (without disclosing title or contents) after the risk has been mitigated and merged.

After a risk has been mitigated and we believe that we have the latest best practices in place, we publish the documentation in our public handbook, internal handbook, and/or in the Access Control documentation.

SECTION: security/identity/access-requests/_index.md

{{% alert title="Not Live Yet" color="warning" %}}
You are viewing a preview of documentation for the future state of GitLab Identity v3 (mid 2024). See the Access Management Policy for the GitLab Identity v2 current state with baseline entitlements and access requests. See the roadmap in the epics gantt chart.
{{% /alert %}}

Access Requests

If a user needs access to an application, there are several approaches:

1. The user's attributes match the existing criteria of a role or organization unit that has already been attached to the Okta application. Access is automatically granted without a request.

1. The organization unit group policy CODEOWNER (ex. division leader or department manager or Executive Business Assistant) can update the policy in accessctl to include the additional role. The manifest of users for the organization unit is automatically recalculated and the users will be added to the organization unit Okta group that has already been attached to the application.

1. The application CODEOWNER can add the additional role group to the application using Terraform.

This provides improved maintenance since the division and department leaders or their delegate (ex. Executive Business Administrator) centrally manage the policies for organization unit groups and which roles are members.

Since the users that are attached to each group are managed by accessctl policies and REST API calls (not Terraform), the changes to Terraform state management are far and few between which

simplifies auditability.

SECTION: security/identity/approvals/_index.md

{{% alert title="Not Live Yet" color="warning" %}}
You are viewing a preview of documentation for the future state of GitLab Identity v3 (mid 2024). See the Access Management Policy for the GitLab Identity v2 current state with baseline entitlements and access requests. See the roadmap in the <a href="https://gitlab.com/groups/gitlab-com/gl-security/identity/eng/-/roadmap?state=all&sort=startdateasc&layout=QUARTERS&timeframerangetype=THREEYEARS&grouppa
tlab-com/gl-security/identity/eng&progress=WEIGHT&showprogress=true&showmilestones=false&milest
onestype=ALL&showlabels=true">epics gantt chart.
{{% /alert %}}

GitOps Workflow

mermaid
gitGraph
  commit id: "Change 1"
  commit id: "Change 2"
  branch change
  checkout change
  commit id:"Current Changes"
  commit id:"CI/CD Validate and Plan Jobs" type: REVERSE
  commit id:"Peer Review" type: HIGHLIGHT
  commit id:"Identity Approval" type: HIGHLIGHT
  commit id:"CODEOWNER Approval" type: HIGHLIGHT
  checkout main
  commit id: "Change 3"
  commit id: "Change 4"
  merge change
  commit id:"CI/CD Terraform Apply Jobs" type: REVERSE
  commit id: "Change 6"

We have a GitLab repository for each vendor instance with a .gitlab-ci.yml file with CI/CD pipeline jobs for terraform validate, checkov (Iac SAST scanning), terraform plan, terraform apply, and terraform destroy jobs.

All changes are performed in GitLab branches that have a terraform validate, checkov, and terraform plan job. Merge requests are configured to require all jobs to succeed, all approvals to be obtained and are merged automatically after all approvals.

Approval Rules

Each merge request requires a peer review and is configured with three (2) GitLab approval rules. The peer reviewer is allowed to add commits to make fixes or make suggestions in merge

request review comments.

1. The Identity Approval approval requires review from the Identity Engineering or Identity Operations team to ensure technical accuracy. This can be performed by the Identity Peer Reviewer if they did not make commits. If the Peer Reviewer makes commits, then an additional person must provide approval for separation of duties.
1. The System Owner approval uses the CODEOWNERS file that specifies the business owner and technical owner for each directory or file in the Terraform GitLab repository